

TOPIC 01: WHAT IS GIT & GITHUB?

What is Git?

Git is a **Free, Open-Source Distributed Version Control System (VCS)** designed to track changes in source code and maintain complete project history over time.

What is GitHub?

GitHub is a cloud-based hosting platform/website where developers store, share, and collaborate on Git repositories online.

GIT (LOCAL TOOL ON PC) → Push to Web →

GITHUB (CLOUD HOST / WEBSITE)

TOPIC 02: FILE STATUS LIFECYCLE

The 4 Core File States:

UNTRACKED (NEW FILE) → git add →

STAGED (READY FOR COMMIT)

↓ git commit

UNMODIFIED (COMMITTED IN REPO) → Edit file →

MODIFIED (UNSTAGED CHANGES)

Standard 3-Step Workflow:

```
$ git add index.html # Step 1: Move from Working Directory to Staging Area $ git  
commit -m "save msg" # Step 2: Snapshot staged files into Local Repository $ git push  
origin main # Step 3: Upload Local commits to Remote (GitHub)
```



Key Rule: git add . stages all modified and newly created untracked files at once!

TOPIC 03: BASIC COMMANDS (CLONE & STATUS)

- `git clone <repo_url>` → Downloads and copies an entire remote repository to your local computer.
- `git status` → Displays the state of the working directory and staging area (shows modified, staged, or untracked files).

TOPIC 04 & 05: NEW REPO & REMOTE SETUP

First-Time Setup Sequence:

```
$ git init # Initialize a new local Git repository $ git remote add origin <repo_url>
# Connect local repo to remote GitHub repository $ git remote -v # Verify remote
connection URLs $ git branch # List local branches $ git branch -M main # Rename
current branch to 'main' $ git push -u origin main # Push commits and set upstream
tracking
```

TOPIC 06: BRANCH COMMANDS

What is a Branch? An independent timeline to develop features safely without affecting production code.

Command	Purpose & Behavior
git branch	Lists all branches (* marks the active branch).
git checkout <branch>	Switches/navigates to the specified branch.
git checkout -b <new_branch>	Creates a new branch AND switches to it immediately.
git branch -d <branch>	Safely deletes a merged branch.



Modern Git Tip: You can also use `git switch <branch>` and `git switch -c <new_branch>!`

TOPIC 07: MERGING CODE & DIFF

- `git diff <branch>` → Compares commits, branches, or unstaged changes side-by-side.
- `git merge <branch_name>` → Integrates and combines the specified branch into the current checked-out branch.

```
$ git checkout main # 1. Switch to destination branch $ git merge feature-login # 2.  
Merge feature branch into main
```

TOPIC 08: GIT PULL ORIGIN MAIN

Pull = Fetch + Merge

Fetches updates from the remote GitHub repository and instantly merges them into your local active branch.

```
$ git pull origin main # Pull teammates' latest commits to keep code synced
```

TOPIC 09: FIXING MISTAKES (UNDOING CHANGES)

Scenario	Problem	Solution Command
Case 1	Accidentally staged a file (Unstage after git add)	git reset <filename> or git restore --staged <file>
Case 2	Undo last commit but keep code in working directory	git reset HEAD~1 (Soft undo: keeps code edits)
Case 3	Roll back multiple commits to a specific checkpoint	git reset <commit_hash>



Caution: git reset --hard permanently deletes uncommitted changes! Use with care.

TOPIC 10: WHAT IS FORKING?

Concept of Fork:

A fork is an independent **personal copy** of another user's public repository stored under your own GitHub account.

ORIGINAL UPSTREAM REPO

→ Fork →

YOUR ACCOUNT COPY (ROUGH COPY)

↓ (Apply fixes & changes)

OPEN A PULL REQUEST (PR) TO UPSTREAM

- Inherits code and visibility settings from the original repository.
- Allows you to freely experiment and contribute to Open Source projects without affecting the main project.